Data-Structures & Algorithms

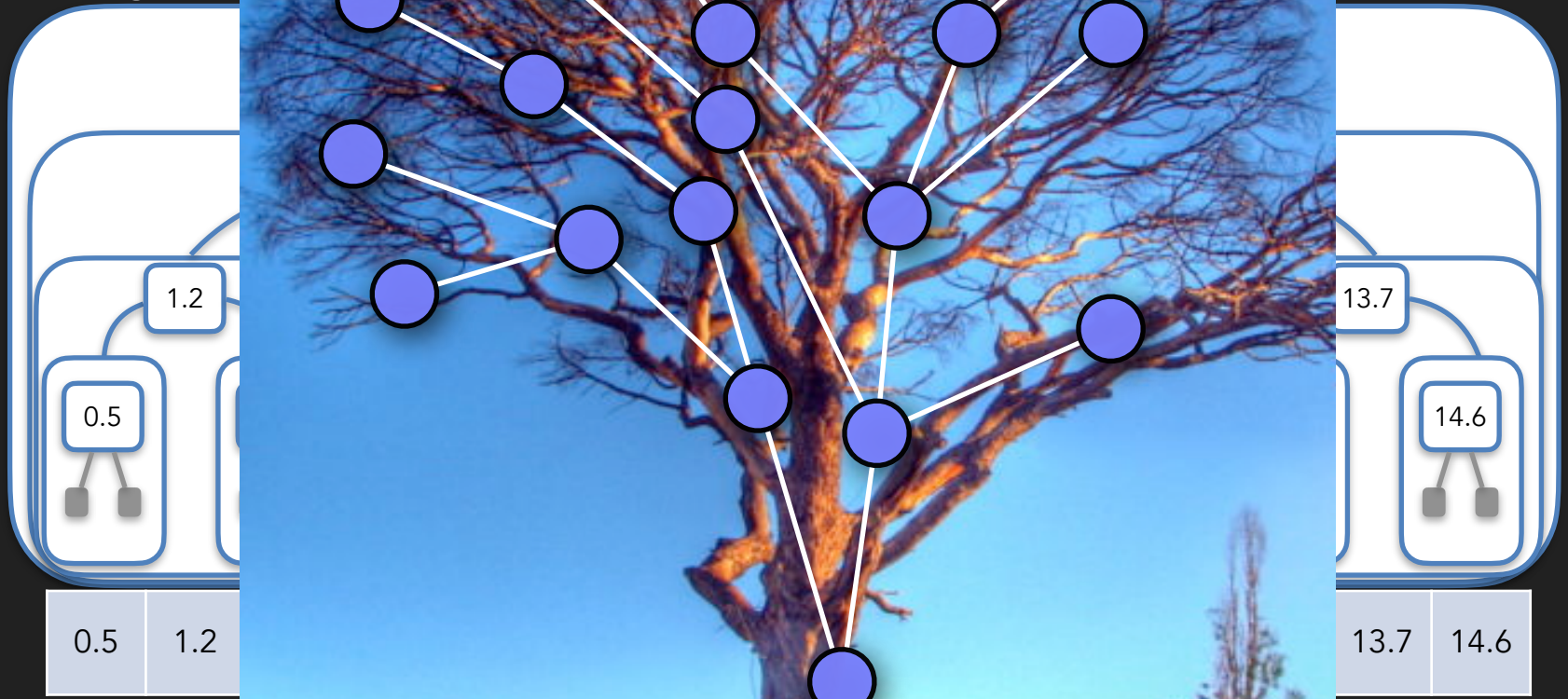
with

Manoj Prabhakaran

Lecture 11

Binary Trees

- Mimicking ep the left and right h



Anatomy of a Binary Tree

- A tree is a collection of "nodes", each of which can have links to other nodes ("child" nodes)
- A tree has no cycles
- A unique node designated as the "root" node
- Every node has a path from the root
  - Path: a sequence of nodes $v_1,\ldots,v_t$ where v_i has a link to v_{i+1}
- Every node other than the root has a unique parent node. Root has none.
- Every node has a unique path from the root
  - Starting form the given node, trace back through the parent at each node, until no parent (can't loop). We must be at the root.

Anatomy of a Binary Tree

- A node could be an internal node (carrying data), or could be an external or leaf node (without any data, and no link going out)
  - For us the leaves will be "missing" (pointer to a leaf is nullptr)
  - The root can be a leaf node (if the tree is empty)
- Each internal node has exactly two child nodes
  - Such a tree is called a *full* binary tree (we consider only full binary trees)
  - Note: any number of children (0, 1 or 2) can be leaves

Anatomy of a Binary Tree

- Depth of a node: How far is it from the root along this unique path (depth of root = 0)
  - Height of the tree = max depth over all nodes (i.e., over all leaves)
- Number of nodes, n = $O(2^{height})$
- But can't say height = O(log n)
  - In fact, in the worst case, height = $\Theta(n)$
    - The tree can look like a linked list!

Anatomy of a Binary Tree

- A complete binary tree is a full binary tree that has all leaves at the same depth
- A complete binary tree of height h has exactly $2^{h+1} - 1$ nodes, of which 2^h are leaves and 2^h-1 are internal nodes
- More generally, a full binary tree with n internal nodes has n+1 leaves

- Base case: n=0 (one leaf)
- Induction step: Consider k≥1. Suppose holds for n ≤ k-1. If n=k, consider a node of maximum depth; both its children are leaves (else it won't be max depth). Consider removing its two children, so that it becomes a leaf. The smaller tree is still full, and has n'=k-1 internal nodes and by induction, n'+1=k leaves. The original tree has one extra leaf (lose one leaf, gain two leaves): so k+1 leaves.

The Node Structure

- A simple implementation (actual STL implementations are more optimised)
- `struct node {int key; node *parent, *left, *right;}`
  - Parent pointer is optional, akin to `prev` in a (doubly) linked list
- A tree object can be a `node*` pointer to its root
  - Could legitimately be `nullptr` (when the tree is empty)
  - Alternately, a sentinel node could be used to point to the root node (to enable cleaner code without branching for insertion and deletion)

Traversing a Binary Tree

- Nodes in a tree can be visited systematically (and printed out as a sequence), using a recursive logic, starting from the root

```
void explore(node* tree) {
  if(tree==nullptr) return;
  std::cout << tree->key << " ";
  explore(tree->left);
  explore(tree->right);
}
```

pre-order

```
void explore(node* tree) {
  if(tree==nullptr) return;
  explore(tree->left);
  explore(tree->right);
  std::cout << tree->key << " ";
}
```

post-order

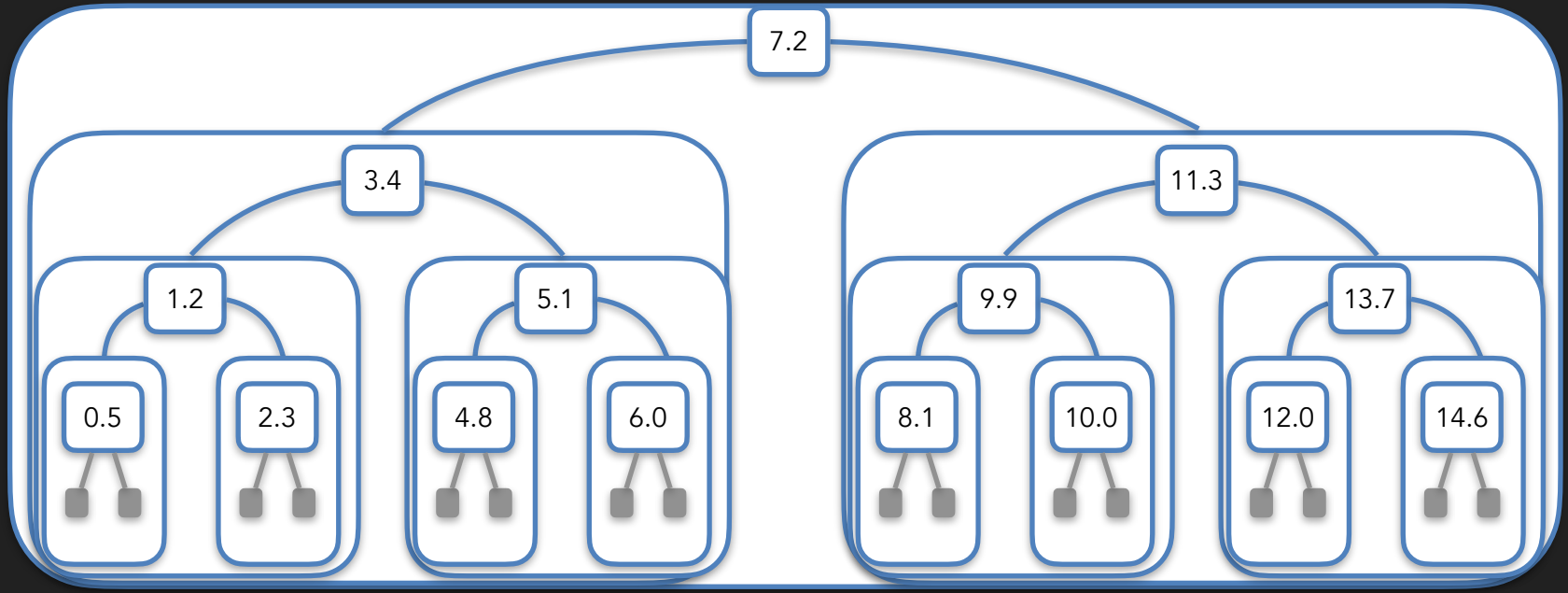
On a tree with n nodes and height h, these traversals take O(n) time, and O(h) extra memory (stack depth)

```
void explore(node* tree) {
  if(tree==nullptr) return;
  explore(tree->left);
  std::cout << tree->key << " ";
  explore(tree->right);
}
```

in-order

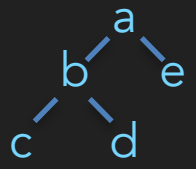
Same as reversed pre-order on mirrored tree

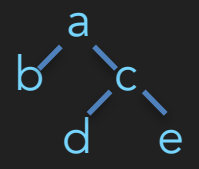
Traversing a Binary Tree

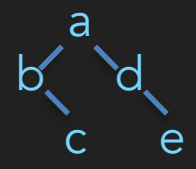


pre-order:	7.2	3.4	1.2	0.5	2.3	5.1	4.8	6.0	11.3	9.9	8.1	10.0	13.7	12.0	14.6
post-order:	0.5	2.3	1.2	4.8	6.0	5.1	3.4	8.1	10.0	9.9	12.0	14.6	13.7	11.3	7.2
in-order:	0.5	1.2	2.3	3.4	4.8	5.1	6.0	7.2	8.1	9.9	10.0	11.3	12.0	13.7	14.6

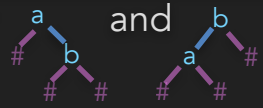
Traversing a Binary Tree

- Traversal sequence loses information about the actual structure of the tree
- E.g., pre-order sequence a b c d e for 

and 

and 

- Can disambiguate using brackets: print "(value *left right*)"
- More compactly: enough to include leaf markers (for pre/post-order)

Not enough for in-order: # a # b # for 


```
void explore(node* tree) {
  if(tree==nullptr) { cout << "# "; return; }
  cout << tree->key << " ";
  explore(tree->left);
  explore(tree->right);
}
```

a b c # # # d # e # #